

Homework 6

Test 1 review

CS3490: Programming Languages

1. Topics to study for Test 1

The test will consist of 15-20 short coding questions, similar in style to the quiz questions. By now, you should have a solid understanding of Haskell programming principles, and of the the standard Haskell types, including

- Lists: `[a]`
- Products: `(a,b)`
- Choices: **Either** `a b`
- Liftings: **Maybe** `a`

You should know both how to take data having these types as input in functions and how to generate them as output. So make sure you know the what each type's data constructors look like, since they determine what cases are necessary in pattern matching.

The test will have 3-4 questions on each of the following topics

- List recursion
- Lambda Calculus and Combinatory Logic
- Higher-order functions: `map`, `filter`, `zipWith`, `foldr`
- Tree recursion
- Encoding of syntax by datatypes — including the `Prop` type from this homework.

2. Warm-up problems

The following problems review list recursion in a context of string processing. You are encouraged to complete these problems before starting the actual homework in the next section, because you will need to use similar kinds of recursion patterns there.

1. `mapAppend :: (a -> [b]) -> [a] -> [b]` applies a function producing lists to every element of the list, and then appends all the lists together.

```
mapAppend show [1,23,456] = "123456"
mapAppend (\c -> c : " ") "hello" = "h e l l o "
mapAppend (\s -> [ take i s | i <- [1..3] ]) ["hello","there","world"]
    = ["h","he","hel","t","th","the","w","wo","wor"]
```

2. `addLetter :: Char -> [String] -> [String]` attaches the given letter in front of every string in the list.

```
addLetter 'x' ["hey","again"] = ["xhey","xagain"]
```

3. `addLetters :: [Char] -> [String] -> [String]` runs the previous function with every character in the given list, and puts the results together in one list.

```
addLetters "abc" ["hey","again"]
    = ["ahey","aagain","bhey","bagain","chey","cagain"]
```

4. `makeWords :: [Char] -> Integer -> [String]` generates every possible word of a given length from a given set of characters.

```
makeWords "abc" 2 = ["aa","ab","ac","ba","bb","bc","ca","cb","cc"]
makeWords "ab" 4 = ["aaaa","aaab","aaba","aabb","abaa","abab","abba",
    "abbb","baaa","baab","baba","babb","bbaa","bbab","bbba","bbbb"]
```

(*Hint.* Use your solution to the previous question.)

5. `update :: (Eq a) => (a,b) -> [(a,b)] -> [(a,b)]` takes a pair `(key,val) :: (a,b)` and a list `lst` of key-value pairs. `update (key,val) lst` returns the list `lst` but with the value associated to key `key` being replaced by `val`. If `lst` does not contain a pair whose first component is `key`, then the pair `(key,val)` should be added to the list.¹

```
> update (2,"Two") [(1,"One"),(2,"2")] = [(1,"One"),(2,"Two")]
> update (2,"Two") [] = [(2,"Two")]
> update ("Y",True) [("X",True),("Y",False),("Z",False)]
    = [("X",True),("Y",True),("Z",False)]
```

¹If it helps, you may assume that `key` only occurs once in the lookup table, but this assumption is not necessary. If `key` occurs more than once, you should update *at least one* occurrence of `key`.

3. Propositional logic

Define the following datatype of Boolean formulas (Propositions):

```
type Vars = String
type Env  = [(Vars,Bool)]
data Prop = Var Vars | Const Bool | And Prop Prop | Or Prop Prop | Not Prop
          deriving (Show,Eq)
```

In the following examples, the single quotes are *backticks*:

```
prop1 = Var "X" `And` Var "Y"           --      X /\ Y
prop2 = Var "X" `Or`  Var "Y"           --      X \/ Y
prop3 = Not (Var "X") `Or` (Var "Y")     --    !X \/ Y
prop4 = Not (Var "X") `Or` Not (Var "Y")  --    !X \/ !Y
```

In the following set of problems, you will implement different functions for analyzing representations of formulas as trees. Among other things, you will implement a function which determines whether a given proposition is *satisfiable*. (The method is essentially the one from discrete math, that proceeds by constructing a truth table.)

You are free at any time to define your own helper functions or to use functions from previous homeworks or lectures.

1. Write a function `fv :: Prop -> [Vars]` which lists all the variables occurring in a given formula. *Each variable should only be listed once.*

(Use a helper function to remove duplicates from the list.)

```
fv prop1 = ["X","Y"]
fv prop2 = ["X","Y"]
fv (prop1 `And` prop2) = ["X","Y"]
fv (Var "X" `Or` Var "Z") = ["X","Z"]
```

2. `countOccurs :: Vars -> Prop -> Integer` takes a variable `x :: Vars` and a formula `p :: Prop` and returns the number of times `x` occurs in the formula `p`.

```
countOccurs "X" (Or (Var "X") (Var "X")) = 2
countOccurs "Y" (Or (Var "X") (Var "Y")) = 1
countOccurs "Z" (Or (Var "X") (Var "Y")) = 0
```

3. Write a function `setTrue :: Vars -> Prop -> Prop` which does the following. Given a variable `v` and a formula `f`, `setTrue v f` outputs the formula `f` in which *every occurrence of `v` is replaced by `True`*.

```
setTrue "X" (Or (Var "X") (Var "Y")) = (Or (Const True) (Var "Y"))
setTrue "X" (Or (Var "X") (Var "X")) = (Or (Const True) (Const True))
setTrue "Y" (Or (Var "X") (Var "X")) = (Or (Var "X") (Var "X"))
```

4. Write a function `lookUp :: Vars -> Env -> Bool`, where `Env` is a type synonym for the list `[(Vars,Bool)]`. Note the capital `U` in the name of the function!

This function takes a key and a list of key-value pairs, and returns the value matching the given key.

If no value is present for a given key, your function can raise an error.

```

lookUp "X" [("X",True),("Y",False)] = True
lookUp "X" [("W",False),("X",True),("Y",False)] = True
lookUp "Y" [("W",False),("X",True),("Y",False)] = False

```

(*Hint.* You may use Haskell's built-in lookup function or write lookUp from scratch using list recursion.)

5. Write the evaluator `eval :: [(Vars,Bool)] -> Prop -> Bool`.

Given a list of variable–value pairs `env`, and a propositional formula `f :: Prop`, the function `eval env f` returns `True` if the formula is true when all the variables are replaced by their values listed in `env`. In writing the evaluator, you should use the standard truth table semantics of `And`, `Or`, and `Not`.

```

eval [("X",False),("Y",True)] prop1 = False
eval [("X",False),("Y",True)] prop2 = True
eval [("X",False),("Y",True)] prop3 = True
eval [("X",True),("Y",False)] prop3 = False

```

6. Write a function `evalList :: Prop -> [Env] -> Bool` which, given a formula `f` and a *list of environments* `envs :: [Env]`, returns `True` if evaluating `f` in *some element of envs* returns `True`. Otherwise, it should return `False`. This function should have type

```

evalList prop1 [("X",False),("Y",True)] = False
evalList prop1 [("X",False),("Y",True)], [("X",True),("Y",True)] = True
evalList prop2 [("X",False),("Y",True)] = True
evalList prop2 [] = False
evalList prop4 [("X",False),("Y",True)], [("X",True),("Y",True)] = True
evalList prop4 [("X",True),("Y",True)] = False

```

7. The last piece you need is a function that, given a list of variables `vars :: [Vars]`, generates the list of *all possible environments* for these variables.

We will split this problem into two.

First, write a function `extendEnv :: [Env] -> Vars -> [Env]` which, given a list of environments and a (new) variable `v`, creates a new list of environments, twice as long as the last one. In the first half, every previous environment is extended/updated by setting `v` to `False`, while in the second half of the output list, the variable `v` is set to `True`.

In other words, the outputted list should extend the environments from the original list by both possibilities for the propositional variable `v`. All resulting possibilities should be outputted in the same list.

```

extendEnv [("X",False),("Y",True)] "Z"
= [("Z",False),("X",False),("Y",True)], [("Z",True),("X",False),("Y",True)]
extendEnv [("X",False)], [("X",True)] "Y"
= [("Y",False),("X",False)], [("Y",False),("X",True)],
  [("Y",True),("X",False)], [("Y",True),("X",True)]
extendEnv [] "X" = [("X",False)], [("X",True)]

```

(*Hint.* This can be programmed in one line using two instances of `map` with correct lambda functions.)

8. Now write the function `genEnvs :: [Vars] -> [Env]` that generates *all* possible environments for a given set of variables.

Intuitively, you should think of this function as generating the “input rows” to the truth table, where every combination of True/False values is assigned to the variables given in the input list. EXAMPLE:

```
genEnvs ["X"]      = [{"X",False}, {"X",True}]
genEnvs ["X","Y"] = [{"X",False}, {"Y",False}, {"X",False}, {"Y",True}], [{"X",True}, {"Y",False}], [{"X",True}, {"Y",True}]]
genEnvs []         = [[]]
```

(*Hint.* Use the previous function as you go through the list of variables recursively.)

9. With the functions above, it is now a straightforward matter to implement `sat :: Prop -> Bool` that checks whether the given formula is *satisfiable*: it returns True if there is some assignment to the variables that makes the formula evaluate to True.

```
sat (Var "X" `And` Var "Y")          = True
sat (Var "X" `And` (Not $ Var "X")) = False
```

Remark. The problem of deciding satisfiability is *NP-complete*. This means that if you find an algorithm that solves this problem and whose running time is polynomially proportional to the size of the input, then you can get \$1,000,000. For more, see the wiki article: https://en.wikipedia.org/wiki/Boolean_satisfiability_problem

10. As another application of the truth-table method, write a function to check whether two formulas are *logically equivalent*.

Write the function `checkEq :: Prop -> Prop -> Bool` that, given two formulas `f1` and `f2`, returns True if they are equivalent.

That is, `checkEq f1 f2` returns True if, for *any* truth assignment `env`, it is the case that `eval env f1 == eval env f2`.

```
checkEq prop1      prop2 = False
checkEq prop3      prop4 = False
checkEq (Not prop1) prop4 = True
checkEq (Or prop2 prop3) (Const True) = True
```

(*Hint.* When generating all environments, include the variables from *both* formulas. That is, you should call `genEnvs` on the list of variables that occur in *either* of the two formulas.)